

ChaTree 项目报告

一、程序功能介绍

我们的项目名为 ChaTree，源自同学们在日常 AI 对话中遇到的实际问题：追问打乱原文阅读节奏，对话结构碎片化、缺乏整体感。本项目将传统 AI 对话与笔记管理软件（如 Obsidian）的部分功能相结合，打造综合性 AI 对话管理平台，涵盖树状文件夹、消息链接、知识图谱等功能。

对话管理：树形文件夹组织对话，支持拖拽排序、重命名、删除。流式 token 级实时渲染，Markdown + KaTeX 数学公式排版。输入框左侧 + 按钮或拖拽上传代码/PDF/Word 文件，UI 显示文件名标识，API 携带完整文件内容。支持单对话和全局导出为 .md 格式，在 Obsidian 中打开可保留对话间的逻辑结构并在知识图谱中展示。

追问与支线：选中 AI 回复中任意文字右键发起追问，右侧面板显示 AI 回答。追问可一键扩展为多轮支线对话（无严格 token 限制），支线在右侧面板独立展示，支持手动标签和跨对话链接。支线在知识图谱中以青色菱形节点呈现，连接到主线的对应轮次节点。

消息链接与知识图谱：在不同对话的消息间建立引用链接，消息下方显示正向/反向链接徽章，点击跳转。AI 自动推荐链接功能在每轮回答后分析当前轮次与历史轮次的关联（同对话相隔 ≥ 5 轮），弹窗确认后自动创建。pyvis + QWebEngineView 实现知识图谱可视化（主线圆节点 + 支线菱形节点 + 链接边），热力图按日期统计对话与支线活跃度。

辅助功能：AI 每轮对话后自动生成 3-5 个标签（支持手动增删）；重试编辑（修改上文消息后重新生成助手回复）；暗色全局主题；内置 15+ 模型预设（OpenAI GPT-5/4 系列、Gemini 3、DeepSeek V4、通义千问 Qwen3.7、Kimi K2.5/2.6）。

二、项目各模块与类设计细节

1. 数据模型 (chaTree/models.py)

基于 Python dataclass 定义核心数据结构，全部支持 `to_dict()/from_dict()` JSON 序列化，确保对话数据可持久化存储与跨会话恢复：

- **Conversation** — 对话聚合根：包含 `id`、`title`、`messages`、`tags`、`links`、`branches`。`to_dict()` 递归序列化所有子对象（消息、注解、支线、链接），`from_dict()` 支持向前兼容（旧数据无 `branches` 字段时默认为空列表）
- **MessageNode** — 单条消息：`role` (user/assistant)、`content`、`annotations`、`tags`。用户和 AI 消息共用同一结构
- **Annotation** — 行内批注：记录 `quoted_text`（选中原文片段）、追问问题与 AI 回答。`branch_id` 字段指向扩展支线（空串表示未扩展），采用 Composition 模式避免循环引用
- **Branch** — 支线对话：独立的 `messages` 列表 + `tags` + `links`，通过 `annotation_id` 和 `source_msg_id` 挂载到主对话的特定轮次，整体作为单个节点出现在知识图谱中
- **Link** — 跨对话引用：`source_msg_id` $\rightarrow$ `target_conv_id` + `target_msg_id`，支持 `selected_text` 存储锚点文本
- **Folder** — 递归文件夹树：`subfolders: list[Folder]` 实现无限层级嵌套，`conv_ids` 存储直接包含的对话 ID

2. 工作区 (chaTree/workspace.py)

Workspace 为全局单例 `ws`，是整个应用的状态管理中心：

- **设置管理**： `api_key`、 `base_url`、 `model`、 `auto_link_suggestions` 等配置项， `save_settings()` 触发 `_save_index()` 写入 `index.json`
- **数据聚合**： `conversations: dict[str, Conversation]` 在启动时从 `workspace_data/conversations/{id}.json` 批量加载， `folders: list[Folder]` 维护文件夹树
- **反链索引**： `_backlink_index: dict[tuple, list[Link]]` 在每次 `save_conversation()` 后通过 `_rebuild_backlink_index()` 重建，遍历所有对话及其支线内的 `links`，支持 `get_backlinks()` 和 `get_outgoing_links()` 双向查询
- **搜索**： `search(keyword)` 遍历标题、标签、消息内容，返回结构化匹配结果 `list[(Conversation, MessageNode, info)]`
- **CRUD**： `add_conversation()`、 `delete_conversation()`、 `move_conversation()`（拖拽移动）、 `add_folder()`（支持嵌套）

3. 后台 Worker (`chaTree/workers.py`)

所有 LLM 调用封装为 `QThread` 子类，通过 Qt Signal 与 UI 层异步通信，避免阻塞主线程：

Worker	类型	核心逻辑
ChatWorker	流式	主对话生成， <code>token_received</code> 逐 token 发射，UI 层 80ms 定时器防抖刷新。流式遍历跳过 <code>chunk.choices</code> 为空的结束块（兼容千问等 API）
AnnotationWorker	流式	批注生成， <code>max_tokens=512</code> ，完成后按 200 字符智能截断（在句号/换行等边界处断开）
BranchWorker	流式	支线对话生成，结构与 ChatWorker 一致，使用独立的 <code>BRANCH_SYSTEM_PROMPT</code> ，无字符截断限制
TagWorker	非流式	调用 LLM 生成标签，解析逗号/顿号分隔的字符串，截取前 5 个。对 <code>reasoning</code> 模型空 <code>content</code> 做兼容处理
AutoLinkWorker	非流式	构建候选轮次列表（同对话跳过当前 +-5 轮，跨对话取全部 user 消息）→ LLM 分析关联 → 解析 JSON → 与已有链接去重 → 截断至 3 条

每个 Worker 均通过 `_prepend_system()` 注入对应的 System Prompt（集中定义在 `prompts.py`）。

4. UI 核心 — ChatPanel (`chaTree/ui/chat_panel.py`)

ChatPanel 是系统中最复杂的 UI 组件，管理消息收发的完整生命周期：

```
send_message() → 构建 MessageNode → append_message + start_streaming
                → ChatWorker 启动（独立线程）
                → _on_token: _buf 累积 token → 80ms _flush_stream
                → _on_done: finalize_streaming + 保存 + 标签生成 + 自动链接检查
```

关键设计决策：

- **双渲染路径**: `USE_WEBENGINE=True` 使用单一 `ConversationView` (`QWebEngineView`, JS bridge 实现流式更新), `False` 回退到多个 `MessageBubble` widget
- **延迟 streaming**: 重试时 `reload_all` 异步加载新页面后, 通过 `_pending_streaming_id + loadFinished` 回调延迟启动 streaming, 避免 JS 执行到旧页面上
- **文件上传**: + 按钮与拖拽 (`dragEnterEvent/dropEvent`) 双入口, `_attached_files` 去重存储。`display_text` (UI 气泡显示文件名标识) 与 `api_text` (API 消息含完整文件内容) 分离
- **自动链接**: `_on_done` 后触发 `_start_auto_link_check()` → `AutoLinkWorker` → `_on_auto_link_suggestions()` → `LinkSuggestionDialog` 弹窗确认 → 创建 Link → 刷新视图

5. WebEngine 视图 (`chaTree/conversation_view.py`)

单一 `QWebEngineView` 渲染整个对话列表, 通过 JavaScript bridge 实现 Python ↔ JS 双向通信:

- **增量更新**: `append_message()` 通过 `insertAdjacentHTML` 追加消息, `start_streaming()` 插入流式占位 div, `append_token()` 调用 `ChainTree.updateStreaming()` 更新内容, `finalize_streaming()` 将占位替换为最终 HTML
- **导航拦截**: 自定义 `_ConvInterceptorPage.acceptNavigationRequest()` 拦截 `ann://`、`link://`、`action://` 等 URL scheme, 将点击事件路由到 Python 侧处理 (注解跳转、链接跳转、重试/标签等操作)
- **延迟操作队列**: `_on_load_finished` 回调依次处理 `_pending_streaming_id`、`_pending_scroll_msg_id` 等延迟操作, 解决 `setHtml()` 异步加载的时序问题
- **KaTeX 渲染**: 每次 DOM 更新后调用 `renderMathInElement()` 对新增内容做数学公式渲染

6. 辅助面板与可视化

- **MainWindow** (`main_window.py`): 三栏布局 (`Sidebar | ChatPanel | QStackedWidget`), 右侧面板默认隐藏, 通过 `annotation_shown/branch_created/branch_requested` 等信号驱动 `AnnotationPanel` 与 `BranchPanel` 的切换
- **BranchPanel** (`branch_panel.py`): 支线多轮对话的完整 UI (~350 行), 含消息列表、标签管理、链接创建、Ctrl+Enter 发送, 使用 `WebEngineContentView` 渲染带 LaTeX 支持的原始批注回答
- **AnnotationPanel** (`annotation_panel.py`): 批注展示面板, 含"扩展为支线"按钮。点击时检查 `ann.branch_id`, 若已扩展则直接导航到支线面板
- **GraphBuilder** (`graph/graph_data.py`): `build()` 返回 `tuple[Network, set[str]]` (pyvis 网络 + 支线节点 ID 集合)。主线轮次为圆形蓝色节点, 支线为青色菱形节点, 链接边为橙色虚线
- **HeatmapDialog** (`heatmap_dialog.py`): 遍历所有对话及支线消息, 按日期聚合消息计数, 自定义 `HeatmapWidget` 绘制色块矩阵

7. 工具模块

- `prompts.py`: 集中管理 5 套 System Prompt (CHAT / ANNOTATION / BRANCH / TAG / AUTO_LINK), 统一 token 限制常量
- `markdown_render.py`: Markdown→HTML 转换, LaTeX 数学公式先提取为占位符再还原, 防止 markdown 解析器破坏公式结构
- `file_reader.py`: 多格式文件读取 (纯文本/PDF/DOCX), 扩展名→emoji 图标 + Markdown 代码块语言标记映射
- `model_presets.py`: `ModelPreset` dataclass (`display_name / model_id / base_url`), 内置 15 个预设, `preset_index_for_workspace()` 根据已保存配置匹配列表选中行
- `styles.py`: 全局 Qt 样式表常量 (按钮、输入框、标签、对话框等深色主题组件)

三、小组成员分工

- **王晨锦**：负责项目总架构，对话、链接、图谱、热力图、导出等大多数功能实现，编写最终报告
- **张云琦**：负责对话前端、API 调用部分，负责演示视频的制作
- **张耀达**：负责用户数据持久化、编写项目阶段性进展报告

四、项目总结与反思

总结

ChaTree 将线性 AI 对话扩展为多维知识网络，解决了一些同学们在学习生活中遇到的 AI 对话管理难的问题，帮助同学们借助 AI 打造自己的知识库。项目涵盖了从数据模型设计、多线程后台任务、WebEngine 前端渲染到知识图谱可视化等多个技术层面，锻炼了我们团队成员的协作能力和技术实力，给我们了解和体验一个完整项目的工作流程提供了宝贵契机。

反思

- 项目架构：我们最初坚持使用轻量化的 QTextBrowser，但无论如何修补都无法很好支持 LaTeX 数学公式，最终选择了 QWebEngineView + KaTeX 的方案才解决了这个问题。这让我们认识到技术选型应优先考虑功能可行性而非体量。
- 知识图谱设计：我们最初对这个功能的实现有畏难情绪，觉得实现类 Obsidian 的动态图谱过于困难。实际开发中发现利用已有的 pyvis 库，编写节点结构和图谱连接方式即可实现，难度远低于预期。
- 前端协作：我们最初的前端设计有两个同学各写了一版，没有开始阶段统一风格和模板，导致后续合并时出现不一致。应该在项目初期就确立统一的设计规范。

后续方向

- 针对于 Coding 的功能特化：可以更结构化地学习编程，实时编写、运行 AI 生成的代码，用类似的结构管理大型工程项目，使代码结构清晰化
- 更多样的输入形式支持：如图片输入、语音输入等，以及更丰富的输出形式（图表、流程图等）
- 用户反馈驱动迭代：积极收集用户反馈和建议，不断改进和完善项目功能，提升用户体验